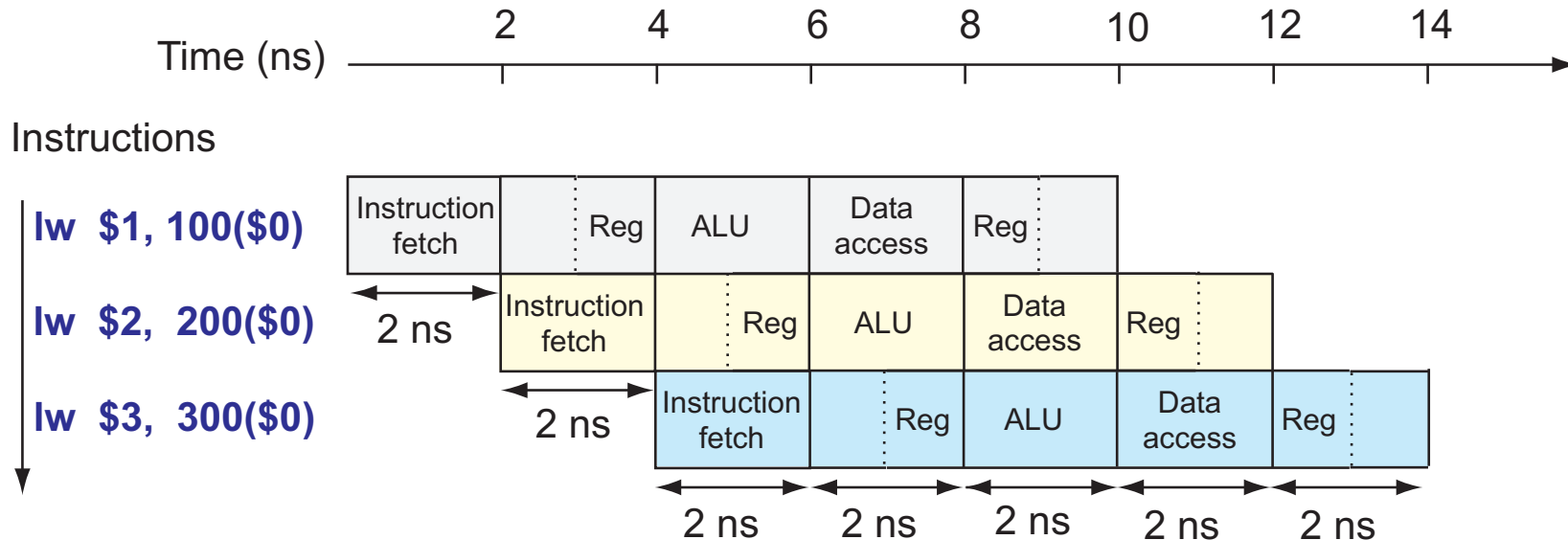
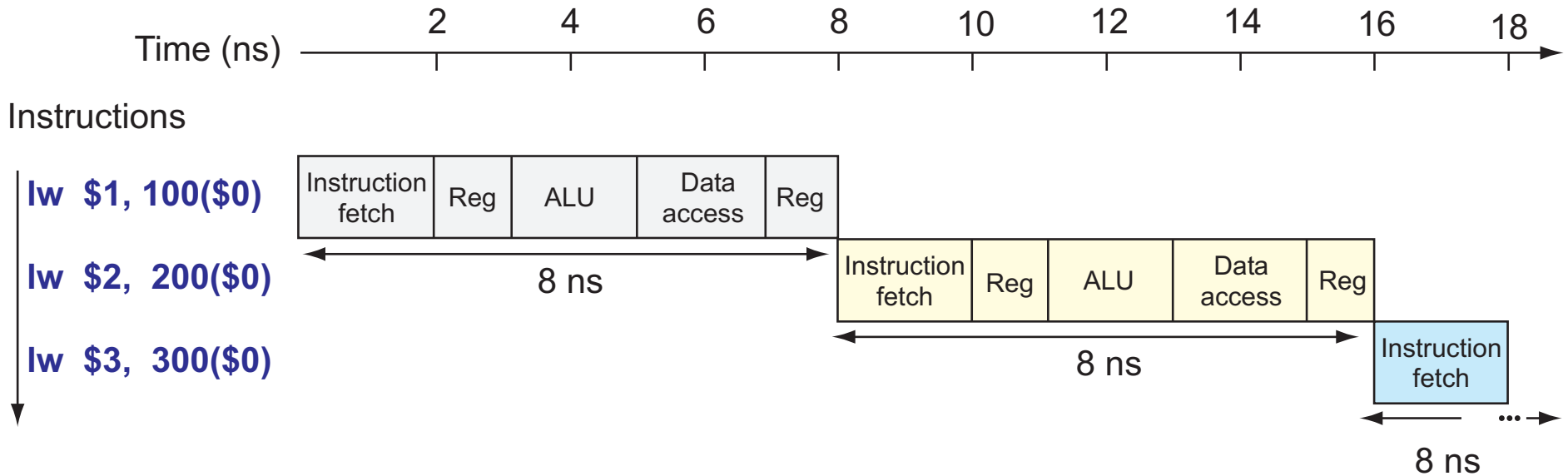


Pipeline

- **Técnica de implementação:**
 - Para processador e unidades funcionais
- **Idéia:**
 - Dividir execução da instrução em passos:
 - **Estágios** do pipeline
 - Cada estágio leva 1 ciclo do clock
 - Inicia uma nova instrução a cada ciclo
 - Sobreposição de execução de múltiplas instruções ⇒
Explora **ILP – Instruction Level Parallelism**
- **Objetivo:**
 - Melhorar desempenho
- **Vantagem:**
 - Não é visível para programador
- **Desvantagem ?**

Exemplo 1: Execução no MIPS “Monociclo” × Pipeline



Desempenho das Implementações do Processador MIPS

- **Datapath “monociclo”:**
 - Cada instrução leva 1 ciclo do clock
 - Tempo do ciclo: determinado pela **instrução** mais longa = 8 ns
 - A cada 8 ns termina uma instrução $\Rightarrow$ Tempo entre instruções = 8 ns
 - **CPI** =
- **Pipeline:**
 - Cada estágio do pipeline leva 1 ciclo do clock
 - Tempo do ciclo: determinado pelo **estágio** mais longo = 2 ns
 - Todas as instruções passam pelos 5 estágios $\Rightarrow$
Tempo de execução de uma instrução = $5 \times 2 \text{ ns} = 10 \text{ ns}$
 - A cada 2 ns termina uma instrução $\Rightarrow$ Tempo entre instruções = 2 ns
 - **CPI ideal** =
- **Tempo de execução de uma instrução:**
 - Não melhora no pipeline (não reduz) e pode aumentar
- **Throughput:** N^0 de instruções terminadas por unidade de tempo
 - Melhora no pipeline (aumenta)

Desempenho do Pipeline

- **Condições ideais:**

- Estágios do pipeline são balanceados
- Há instruções suficientes para executar $\Rightarrow$ Transiente inicial desprezível
- Não há conflitos
- Não há overhead de controle

- **Em condições ideais:**

$$speedup_{pipeline} = n^0 \text{ de } estágios_{pipeline} = 5$$

- **Em condições quase ideais:**

- Transiente inicial desprezível, sem conflitos, **estágios desbalanceados**

$$Speedup \cong \frac{\text{tempo entre instruções}_{sem\ pipeline}}{\text{tempo entre instruções}_{com\ pipeline}} = \frac{8}{2} = 4 < 5$$

- **Por que não ter pipeline com MUITOS estágios ?**

Conflitos (Hazards) no Pipeline

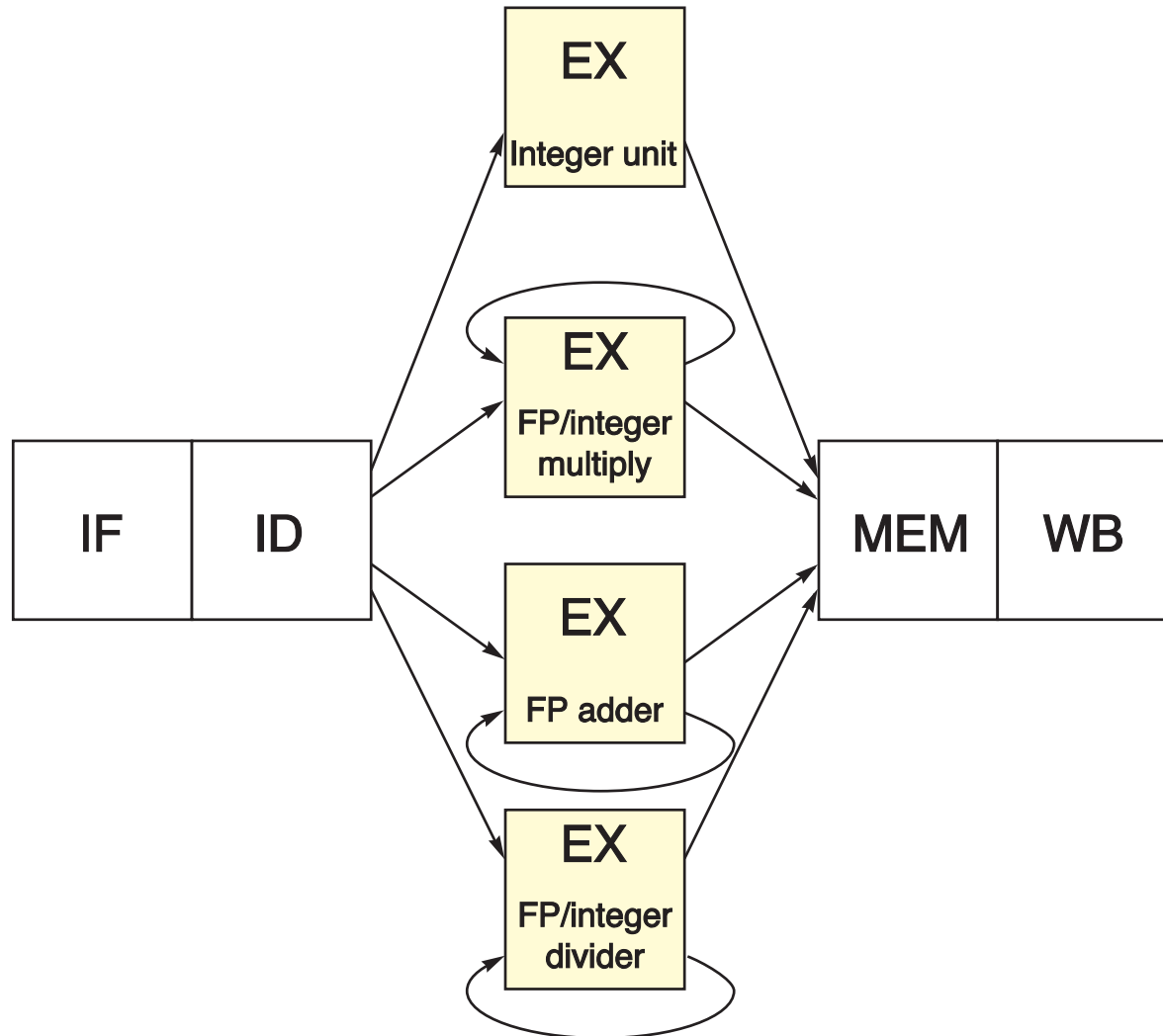
- Situações que impedem que próxima instrução execute no próximo ciclo
- Podem causar atrasos na execução das instruções $\Rightarrow$
Reduzem desempenho $\Rightarrow$ Speedup do pipeline será menor
- **Tipos de conflitos:**
 - **Estruturais:**
 - Conflito por recurso de hardware
 - Soluções: **Replicação do recurso, stall**
 - **De dados:**
 - Instrução usa resultado de outra instrução que ainda está no pipeline
 - Soluções: **Forwarding** (bypassing), **stall**, **escalonamento** de instruções
 - **De controle:**
 - Instrução deve ser ou não executada, dependendo do resultado de outra que ainda está no pipeline
 - Desvios condicionais e incondicionais
 - Soluções: **Predição, delayed branch, stall**

Pipeline com Operações Multiciclo

- Operações que consomem mais de 1 ciclo no estágio EX
- **Exemplo:**
 - Operações de multiplicação e divisão de inteiros
 - Operações de ponto flutuante
- **Solução:**
 - Estágio EX pode ser repetido várias vezes para completar operação
 - Várias unidades funcionais (pipelined ou não)
- **Problemas:**
 - Conflitos estruturais
 - Solução: **Intervalo de iniciação**
 - Conflitos de dados

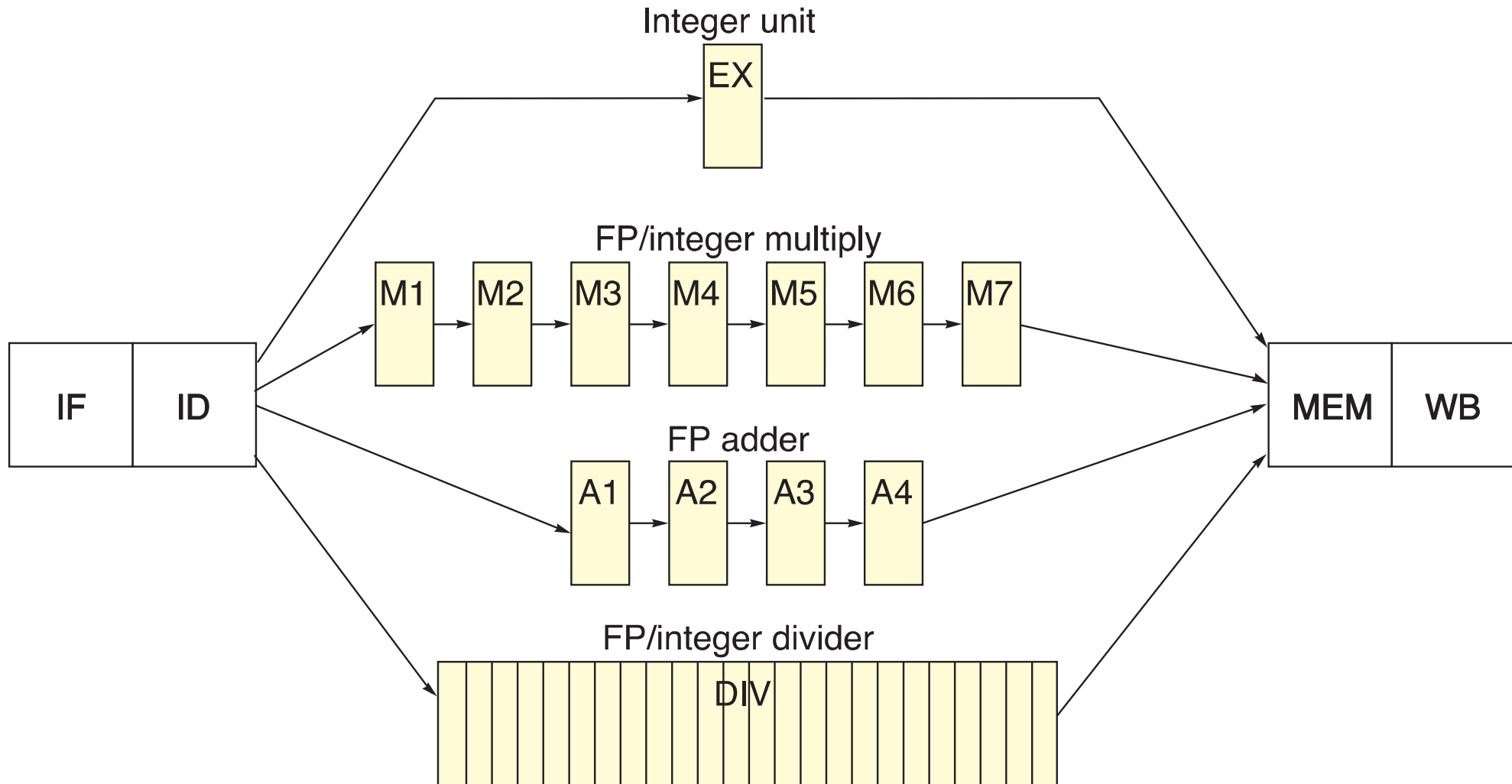
Exemplo 2 (a): Pipeline com Operações Multiciclo

- 3 unidades funcionais adicionais: “non-pipelined”



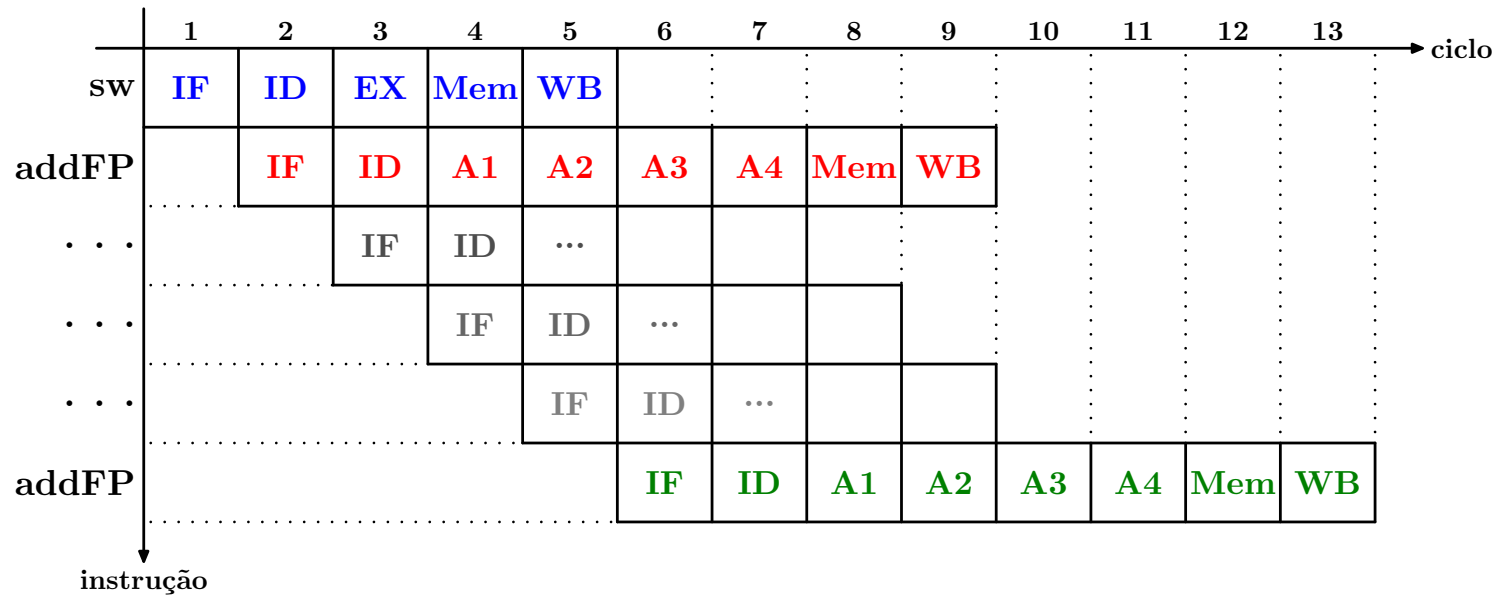
Exemplo 2 (b): Pipeline com Operações Multiciclo

- 3 unidades funcionais adicionais: “**pipelined**”



Exemplo 2: Pipeline com Operações Multiciclo

(a) Com unidades funcionais “**non-pipelined**”: **Intervalo de iniciação** = 4 ciclos



(b) Com unidades funcionais “**pipelined**”:

